INCIDENT DIAGNOSTIC & REMEDIATION REPORT

GitHub Actions Deployment Failure Analysis & Resolution

Complete technical diagnosis, SSH key synchronization, and Utho VM setup guide for `LexvraInfinology/dao`

REPOSITORY	WORKFLOW FILE	FAILING STEP	EXIT STATUS
LexvraInfinology/dao	.github/workflows/ci-cd.yml	Deploy and verify on Utho VM	Process exit code 255

1. Incident Overview & Log Analysis

Exact Terminal Failure Observed:

```
***@***: Permission denied (publickey,password).
Error: Process completed with exit code 255.
```

During the automated CI/CD pipeline run on GitHub Actions, the **CI (Build & Lint)** job successfully finished. However, the downstream deployment job `Deploy to Utho VM` halted immediately upon attempting an SSH remote connection into the target Utho cloud instance.

The workflow triggers the following OpenSSH command:

```
ssh \
  -i "$RUNNER_TEMP/utho_deploy_key" \
  -p "$CLEAN_PORT" \
  -o BatchMode=yes \
  -o IdentitiesOnly=yes \
  -o StrictHostKeyChecking=accept-new \
  -o UserKnownHostsFile="$HOME/.ssh/known_hosts" \
  "$CLEAN_USER@$CLEAN_HOST" "DEPLOY_BRANCH=${GITHUB_REF_NAME} bash -s" < scripts/deploy/utho-vm.sh
```

2. Root Cause Technical Breakdown

The SSH exit code `255` paired with `Permission denied (publickey,password)` indicates that the SSH daemon (`sshd`) running on the Utho VM strictly rejected authentication. The failure stems from three specific factors:

Component	Current State	Impact
Utho VM `authorized_keys`	Public key is missing from the server's `~/.ssh/authorized_keys` file.	The server cannot verify incoming authentication challenges against the private key.
GitHub Secret: `UTHO_SSH_PRIVATE_KEY`	Contains either an outdated key, mismatched key, or improper newline/carriage-return characters.	OpenSSH presents an identity that does not match any allowed public key on the host.
OpenSSH Flag: `BatchMode=yes`	Enforced in line 121 of `ci-cd.yml`.	Suppresses interactive password fallback prompts in non-interactive CI, failing immediately with 255.

> **Key Discovery:** An ED25519 deploy keypair already exists in your workspace root (`utho_deploy_key` and `utho_deploy_key.pub`), created with comment `github-actions-utho-deploy`. Authorizing this public key on the

> Utho server resolves the issue.

3. Step-by-Step Resolution Guide

1 Authorize the Public Key on Utho VM

Log in to your Utho Cloud VM via your cloud provider web terminal or an existing root shell, then run this single block to install and lock down the authorized key:

```
# Ensure ~/.ssh folder exists with 700 permissions
mkdir -p ~/.ssh && chmod 700 ~/.ssh

# Append the public key (from utho_deploy_key.pub)
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE4jc4B3Xha2FMb6vKayDTbTws+gFnh/ws1bNOjzUkXQ github-action
s-utho-deploy" >> ~/.ssh/authorized_keys

# Enforce secure permissions required by OpenSSH
chmod 600 ~/.ssh/authorized_keys

# Verify it was added successfully
tail -n 2 ~/.ssh/authorized_keys
```

2 Configure GitHub Actions Repository Secrets

Go to your GitHub repository: **Settings** → **Secrets and variables** → **Actions**. Ensure each of the following secrets is configured:

Secret Name	Description & Expected Value
`UTHO_SSH_PRIVATE_KEY`	Paste the **entire** contents of `utho_deploy_key`, including header and footer: `-----BEGIN OPENSSH PRIVATE KEY----- ... -----END OPENSSH PRIVATE KEY-----`
`UTHO_HOST`	The public IPv4 address or hostname of your Utho VM (e.g., `103.xxx.xxx.xxx`).
`UTHO_USER`	The user under which the application runs (typically `root` or `ubuntu`).
`UTHO_SSH_PORT`	`22` (or your custom SSH port if modified).
`UTHO_KNOWN_HOSTS`	Output of `ssh-keyscan -H <UTHO_HOST>` to prevent man-in-the-middle rejection.

3 Prerequisites on Utho VM for Application Deployment

As defined in `scripts/deploy/utho-vm.sh`, the server must have the application directory pre-initialized:

```
# Ensure destination directory exists and is a git clone
mkdir -p /opt/dao
cd /opt/dao

# If not already cloned:
# git clone https://github.com/LexvraInfinology/dao.git .

# Ensure production .env exists with production values
cp .env.example .env  # (and update DB passwords and RPC settings)

# Verify Docker and Docker Compose v2 are installed
docker compose version
```

4 Trigger Pipeline & Verification

Once the steps above are applied:

1. Go to the failed GitHub Actions run page.
2. Click **Re-run failed jobs** (or trigger a push to `main` / `developer`).
3. The job will authenticate via SSH key, execute `utho-vm.sh`, build the Docker stack, and confirm healthy startup across all 6 microservices: `frontend`, `api`, `indexer`, `queue`, `postgres`, and `redis`.

> **Resolution Complete:** Following Steps 1 and 2 will immediately clear the `Permission denied (publickey,password)` exit code 255.
